

Trajectory-aware Smears for Stylized 3D Animations

Lou Tremolieres

Univ. Bordeaux, CNRS, Bordeaux INP, LaBRI
Bordeaux, France
Inria
Bordeaux, France
lou.tremolieres@free.fr

Pierre Bénard

Univ. Bordeaux, CNRS, Bordeaux INP, LaBRI
Bordeaux, France
Inria
Bordeaux, France
pierre.benard@labri.fr

Jean Basset

Univ. Bordeaux, CNRS, Bordeaux INP, LaBRI
Bordeaux, France
Inria
Bordeaux, France
jean.basset@inria.fr

Pascal Barla

Inria
Bordeaux, France
Univ. Bordeaux, CNRS, Bordeaux INP, LaBRI
Bordeaux, France
pascal.barla@inria.fr

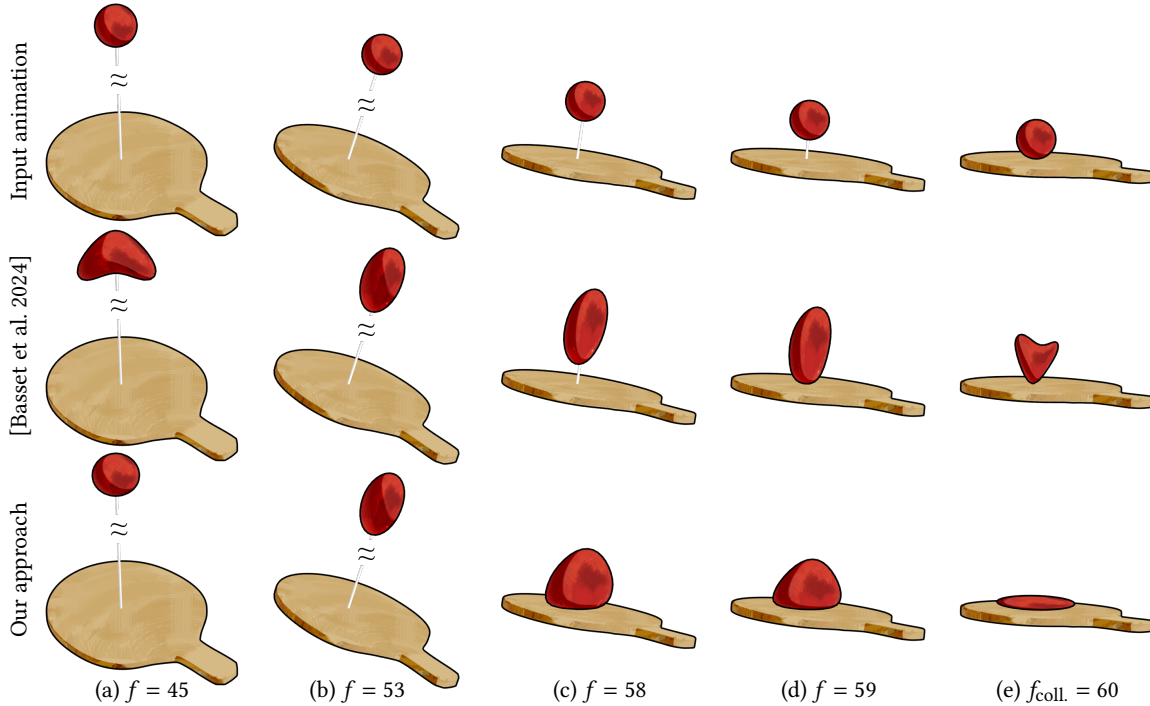


Figure 1: 3D animation of a jokari (top row) stylized with the approach of Basset et al. [2024] (middle row). High trajectory curvatures in (a) and (e) yield unnatural ball deformations. Our approach (bottom row) corrects this issue (a) and adds squash-and-stretch (e) that is easily blended with smearing effects (c and d) to exaggerate animation around the collision.

Abstract

Smearing is an essential effect to expressively convey motion in stylized animations. In this paper, we extend the method of Basset et al. [2024] to better emphasize the main motion's trajectory

of an object when generating elongated in-betweens, i.e., when stretching a 3D object along its trajectory to cover adjacent frames. This limits visual artifacts such as intersections that typically occur when trajectories self-overlap due to local rotations or abrupt changes of direction (trajectories with high curvatures or even discontinuities at contacts). We address these cases with minor computational and memory overheads, and offer enhanced impact expressiveness by combining smear and squash-and-stretch effects at collisions.



This work is licensed under a Creative Commons Attribution 4.0 International License.
MIG '25, Zurich, Switzerland

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2236-3/25/12
<https://doi.org/10.1145/3769047.3769057>

CCS Concepts

• **Computing methodologies** → **Animation; Non-photorealistic rendering.**

ACM Reference Format:

Lou Tremolieres, Jean Basset, Pierre Bénard, and Pascal Barla. 2025. Trajectory-aware Smears for Stylized 3D Animations. In *The 18th ACM SIGGRAPH Conference on Motion, Interaction, and Games (MIG '25), December 03–05, 2025, Zurich, Switzerland*. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3769047.3769057>

1 Introduction

Comic artists and traditional 2D animators rely on motion effects such as motion lines and smear frames to convey and enhance the depiction of fast moving objects. Among those effects, *elongated in-betweens*, which consist in stretching an object along its trajectory to cover the span of several frames, are especially used in 2D animation [Williams 2001]. They have lately regained popularity in stylized 3D productions (e.g., the “Spider-Verse” movies, the “Arcane” series). The recent work of Basset et al. [2024] successfully automatizes the creation of smearing effects for key-framed animations while providing extended artistic control. However, in this method, smearing is applied irrespective of motion trajectory properties. As later shown in this paper, this limits expressivity: deformations are not adapted to convey highly curved trajectories, or even trajectory discontinuities at contacts; and trajectories of self-rotating objects are so convoluted that smearing hinders motion legibility. An important consequence of ignoring trajectory characteristics is that smearing tends to produce self-intersections when the input motion rapidly changes direction.

In this paper, we address these limitations for objects moving rigidly in 3D space (deformable bodies and articulated characters are left for future work). Our key idea is to analyze the trajectory of the object and tune the elongated in-between generation accordingly, which has the positive side-effect of strongly limiting self-intersections. In Section 3, we identify and handle three main problematic cases: (1) rotations of the object around itself (self-rotations), (2) trajectories with high curvatures, and (3) abrupt motion discontinuities at contacts. In the latter case, we additionally introduce some squash-and-stretch to amplify the collision effect. Section 4 and the accompanying video demonstrate the benefits of our method on various test scenes, included in supplemental material with our Blender implementation. We discuss in Section 5 the limitations of our approach and avenues for future work.

2 Related Work

Motion effects. A variety of stylization techniques are aimed at conveying legible fast motion by manipulating the silhouettes of animated objects (e.g., see [Cutting 2002; Lasseter 1987]).

One approach to stylize the silhouette of an object’s motion is to use an approximation of a swept volume (i.e., the space spanned by a 3D object during its animation) to produce artistic motion blur or speed lines [Huang et al. 2012; Jones and Keyser 2005; Schmid et al. 2010]. However, these methods are either too slow to provide interactive feedback, or are computed at rendering time, making their control during the animation process difficult.

Basset et al. [2024] introduced a 3D method to convey convincing motion effects, in particular elongated in-betweens, whereby each vertex of an animated object is stretched along its space-time trajectory. However, this approach suffers from a major limitation: with complex motions such as local rotations around the object centroid or abrupt changes in directions, the elongated in-betweens severely self-overlap, hindering motion legibility as the silhouette of the object’s motion is not clearly conveyed anymore. Our method precisely addresses this limitation.

Farhat et al. [2025] propose an alternative method to create motion effects based on motion ribbons that are attached to the trailing edge of the object instead of individual vertices. This ensures that the silhouette of the object is preserved after stylization, but it is not adapted to the generation of elongated in-between 3D shapes. They also decompose the animation into a hierarchy of transformations and let the artist select only the ones that represent the motion of the object’s silhouette. We borrow this idea in our approach to cancel self-rotations (rotations of an object around its centroid) when computing trajectories.

Exaggerating impacts. Williams [2001] describes a method to stylize impacts, which uses a combination of smearing to make the animated object reach the point of impact earlier, and squash-and-stretch to exaggerate the impact itself. Following this idea, we use squashing in our method to amplify impacts and improve the motion’s readability, while fully integrating it with smearing.

Many methods rely on physical simulations to convey the deformations and secondary dynamics resulting from an impact [Coros et al. 2012; Garcia et al. 2007; Müller et al. 2005; Zhang et al. 2020]. While these approaches give very convincing results, they are not easily applicable to key-framed animations and are difficult to control for artists. Cheney et al. [2002] use a physical simulation to determine a single squashing factor used for deformation. The animated object is squashed along one axis (the forward direction of the animation) and stretched along two other orthogonal axes to preserve volume. This simple approach gives convincing results and has already been used in an art-directed stylization method [Ma et al. 2022] by letting the user control the squashing factor. We follow a similar approach at the impact frame, but combine squashing and smearing for neighboring frames.

3 Method

Our method follows the general pipeline of Basset et al. [2024] for *single rigid objects* (Section 3.1 of their paper); articulated characters are left to future work. Starting from a key-framed animation of a rigid object, we first identify in a pre-process which parts of the object should be leading in front, and which parts should be trailing behind, based on the velocity of the object centroid. More precisely, we compute *motion offsets* $\delta \in [-1, 1]$ for every vertex i and frame f based on the distance of the vertex position $\mathbf{p}_i$ to the plane orthogonal to the normalized velocity $\hat{\mathbf{v}}$ of the object passing through its centroid $\mathbf{c}$:

$$\bar{\delta}_i = \frac{\delta_i}{\max_j |\delta_j|} \quad \text{with } \delta_i = (\mathbf{p}_i - \mathbf{c}) \cdot \hat{\mathbf{v}}, \quad (1)$$

normalizing by the maximum distance among all vertices.

At run-time, elongated in-betweens are created by displacing every mesh vertex according to its motion offset along its trajectory approximated by Catmull-Rom splines [Catmull and Rom 1974]. For artistic control purposes, the displacement of vertex i at frame f , denoted $\beta_i(f)$, is governed by a general elongation stylization function $\mathcal{S}_E$ of motion offsets and arguments “args”:

$$\beta_i(f) = \mathcal{S}_E(\delta; \text{args}). \quad (2)$$

An elongated in-between covering the previous and next frames (e.g., Figures 2b & 2d) is obtained with the default elongation stylization function $\mathcal{S}_E(\delta; \beta_{\max}) = \delta\beta_{\max}$, with $\beta_{\max} = 1$.

3.1 Canceling of rotations

Smearing effects aim at depicting and emphasizing the main motion trajectory of an object, as this corresponds to motion of the object in the image plane. We assume self-rotations of the object (i.e., around its centroid) should be decoupled from the main motion trajectory and be depicted similarly to the original motion (i.e., as rotations), as these correspond to motion of the object onto itself. A natural consequence of this decoupling is the reduction of self-overlaps of the elongated in-betweens as illustrated in Figure 2.

To address this problem, Basset et al. [2024] proposed a work-around which consists in discarding the rotation component of the object motion, while reintroducing it in UV-space to animate the texture of the object. However this “hack” only gives good results for perfectly symmetric objects where rotations of the object or of its texture are indistinguishable, such as spherical balls or spinning tops. Instead, we address this issue by keeping the self-rotation motion component fixed when creating elongated in-betweens.

In the spirit of the motion decomposition of Farhat et al. [2025], when estimating vertex trajectories, we re-use the self-rotation of the object at the current frame for the previous and next frames, in effect locally canceling variations in self-rotations. This results in the disappearance of self-intersections as shown on the rugby ball animation in Figure 2d. More importantly, as best seen in the supplemental video, only the arcing motion of the whole object is conveyed by the smearing effect, while rotations are depicted similarly to the original motion, especially when texture markings are present (observe the white bands of Figure 2c and Figure 2d).

In practice, to avoid storing a trajectory per vertex for each frame of an animation in memory, we separately store the original trajectories (one per vertex for the full animation) and the local rotation of the object (Euler angles and origin in object space at each frame). At run-time, we reconstruct the sought-for trajectory at frame f by canceling the self-rotations at every other frames, and applying to them the self-rotation at frame f , using the stored origin as the center of rotation.

3.2 Curvature-based weighting

As illustrated in Figures 1 and 4, elongated in-betweens along highly-curved trajectories result in different parts of the elongated object conveying different trajectory directions; this reduces motion legibility and might even introduce unwanted artifacts. To address this issue, our idea is to negatively correlate the magnitude of smearing to trajectory curvature, which once again has the positive side effect of limiting self-overlapping artifacts.

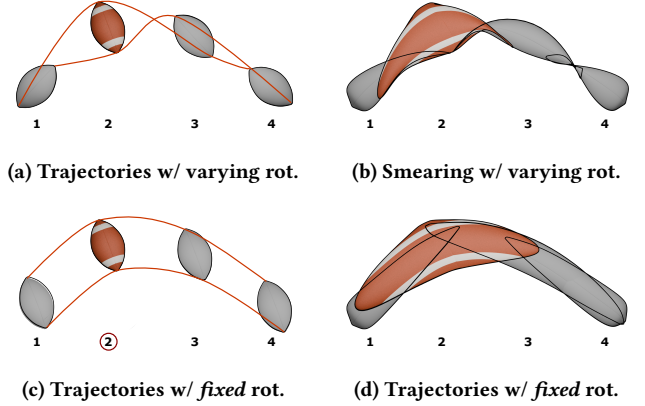


Figure 2: Four frames of a rugby ball animation. Using varying self-rotations, the resulting convoluted vertex trajectories (a) yields unnatural smearing artifacts (b). If we instead keep self-rotations fixed at the current frame (c), smearing only conveys the arc motion, while rotation is still properly depicted, hence reducing self-intersections.

More formally, we introduce a weighting function $\rho_f(t)$ which decreases from 1 to 0 according to the accumulated curvature of the object trajectory from frame f to $t \geq f$ (see Figure 3):

$$\rho_f(t) = e^{-S\left(\int_f^t \kappa(\tau) d\tau\right)}, \quad (3)$$

where $\kappa(\tau)$ is the unsigned curvature of the trajectory of the object centroid at time τ . We use the trajectory of the object centroid since, in most cases, it is a reasonable approximation of the object’s main motion and, assuming that the origin of the rotation in object space is placed at the centroid, it has the advantage of making ρ_f unaffected by local rotations of the object.

The weighting function ρ_f is defined similarly for a time $t \leq f$ by replacing the integration interval by $[t, f]$ in Equation 3. The S function tempers the influence of small accumulated curvatures:

$$S(x) = \begin{cases} x^2(2-x) & \text{if } x < 1, \\ x & \text{otherwise.} \end{cases}$$

We use the ρ_f function to control the length of elongated in-betweens for high-curvature trajectories. Consider for instance the extreme example of a pendulum where the trajectory tangent flips direction at the extremities. Smearing along such a trajectory not only compromises the legibility of the extreme positions, but also creates a geometric artifact as shown in Figure 4 and the supplemental video. We opt to diminish displacement when smearing extends over highly-curved trajectory portions. In order to do so globally over the object, we use a reduced displacement bound $\beta'_{\max}(f) \leq \beta_{\max}$, which we use as part of the arguments of Equation 2. The reduction in displacement magnitude should be proportional to the curvature accumulated along the trajectory. When elongating in the direction of future frames (i.e., $\delta \geq 0$), we use:

$$\beta'_{\max}(f) = \int_f^{f+\beta_{\max}} (\rho_f(t))^a dt, \quad (4)$$

where $a \geq 1$ is a parameter that increases the curvature influence. This is illustrated by the orange area in Figure 3. The case $\delta \leq 0$ is obtained by integrating over the $[f - \beta_{\max}, f]$ interval in Equation 4.

Note that $\beta'_{\max}(f) = \beta_{\max}$ only when $\rho_f(t) = 1$ on the considered time interval, which only occurs if the trajectory is straight. Another special case occurs when the curvature is locally infinite at a time t^* . Assuming the $t > f$ case (i.e., using Equation 3), we then have $\rho_f(t) = 0$ for all $t \geq t^*$, which ensures that $f + \beta'_{\max}(f) < t^*$ (see Equation 4). This approach thus guarantees that smearing will not extend past an abrupt change of trajectory direction. This is particularly relevant for the case of collisions.

3.3 Collision handling

An object colliding with another object usually yields a trajectory discontinuity at the time of contact, which is already handled by the curvature-based weighting scheme exposed in the previous

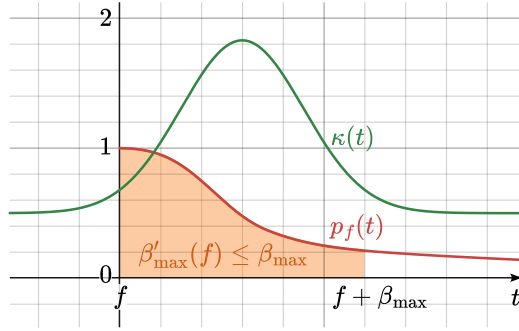


Figure 3: The weighting function $\rho_f(t)$ (in red, here for $t \geq f$) is based on the unsigned curvature $\kappa(t)$ of the trajectory (in green). The curvature-aware displacement bound $\beta'_{\max}(f) < \beta_{\max}$ is the integral (in orange) of $\rho_f(t)$ over the $[f, f + \beta_{\max}]$ range, raised to a user-defined power a (here $a = 1$).

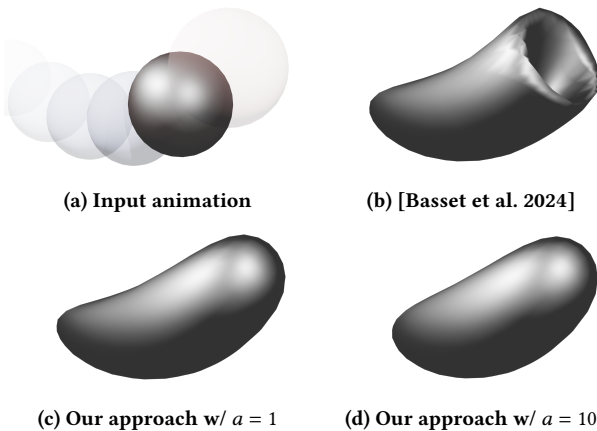


Figure 4: For a pendulum animation (a), the approach of Basset et al. [2024] creates a geometric artifact (here $\beta_{\max} = 2$) where the trajectory takes a sharp turn. Our curvature-based weighting (c) avoids smearing past the sharp turn. Increasing the a parameter (d) further limits smearing due to curvature.

section. In order to increase the legibility of collisions, it is also common in traditional 2D animation to elongate an object so that it makes contact a few frames before and/or after the collision frame (e.g., see [Williams 2001]). We now show how smearing is extended to automatically reproduce such effects, and combine it with an automatic squash-and-stretch deformation to exaggerate contacts.

Collision smearing. In our approach, the collision frame f_{coll} is manually selected by the user, as well as the number of frames m_p (resp. m_n) before (resp. after) f_{coll} , where the stylized object should be elongated to make contact. We then create specific elongated in-betweens for frames before contact (i.e., $f \in [f_{\text{coll}} - m_p, f_{\text{coll}})$) or after contact (i.e., $f \in (f_{\text{coll}}, f_{\text{coll}} + m_n]$). First, we modify the displacements $\beta_i(f)$ so that the smear deformation goes all the way to the collision, regardless of the accumulated curvature (Figure 5b):

$$\beta_i(f) = \begin{cases} \delta_i (f_{\text{coll}} - f) & \text{if } \delta_i > 0 \text{ and } f < f_{\text{coll}}, \\ \delta_i (f - f_{\text{coll}}) & \text{if } \delta_i < 0 \text{ and } f > f_{\text{coll}}, \\ \mathcal{S}_E(\delta_i; \text{args}) & \text{otherwise.} \end{cases}$$

Note that the object at frame f_{coll} is left unaffected by smearing with this approach. In practice, it may be left undeformed, or it may alternatively be squashed and stretched as explained later on.

In order to increase further motion legibility and smoothness around f_{coll} , we propose to progressively blend the smeared objects with the object at collision as it gets closer to it. To this end, we first need to estimate the normal to the impact plane: we simply use the bisector $\mathbf{n}$ between the incoming and outgoing trajectory tangents at f_{coll} , as shown in Figure 5. For each vertex i at position $\mathbf{p}_i$ in a smeared frame, we then blend it with its position at the contact frame using weights $\tilde{d}_i \in [0, 1]$ given by:

$$\tilde{d}_i = \frac{\tilde{d}_{\max} - d_i}{\tilde{d}_{\max} - \tilde{d}_{\min}} \quad \text{with } d_i = \mathbf{p}_i \cdot \mathbf{n}. \quad (5)$$

We use $\tilde{d}_{\max} = d_{\max} + \lambda_{\max} \Delta d$ and $\tilde{d}_{\min} = d_{\min} - \lambda_{\min} \Delta d$ with $d_{\max} = \max_j d_j$, $d_{\min} = \min_j d_j$ and $\Delta d = d_{\max} - d_{\min}$. The parameters $\lambda_{\max} \in [-1, 1]$ and $\lambda_{\min} \in [-1, 1]$ are used to adjust the mass distribution of the blending. They may be tuned differently for frames before versus after f_{coll} .

In the case of a frame f affected by two collisions happening on frames $f_{\text{coll},1}$ and $f_{\text{coll},2}$ ($f_{\text{coll},1} < f < f_{\text{coll},2}$), we compute separately the deformations with regard to $f_{\text{coll},1}$ and to $f_{\text{coll},2}$, and next interpolate between them according to a blending weight $\bar{b}(f) \in [0, 1]$ that captures the relative temporal proximity to the neighbor collision frames. This is done by first computing a temporal proximity $b_k \in [0, 1]$ to the k -th collision frame:

$$b_k(f, m) = \left| 1 - \frac{|f - f_{\text{coll},k}|}{m + 1} \right|,$$

where $\lfloor x \rfloor = 0$ for $x < 0$. The blending weight is then given by:

$$\bar{b}(f) = \frac{b_2(f, m_p)}{b_1(f, m_n) + b_2(f, m_p)}.$$

Squash-and-stretch on contact. Squash and stretch at f_{coll} may be manually performed by the artist. However, for simplicity, we provide an automatic squash-and-stretch operator following the

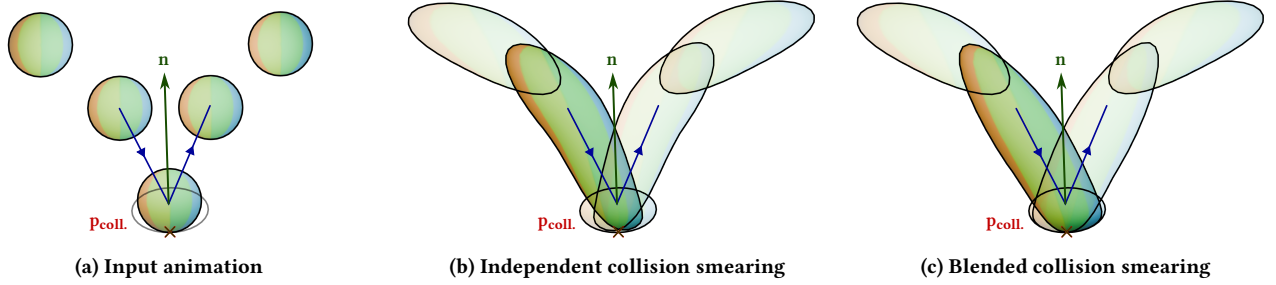


Figure 5: The third frame of an input bouncing ball animation (a) is marked by the user as a collision: we estimate the impact plane normal $\mathbf{n}$ (in green) as the bisector of the linear trajectory (in blue) from frame 2 to 4, and the collision point $\mathbf{p}_{\text{coll}}$ (in red) as the point farthest down this direction. The frame before ($m_p = 1$) and after ($m_n = 1$) collision are first independently modified so that elongated objects make contact (b). Here we also use our automatic squash-and-stretch operator at f_{coll} . Each of the smeared object is then blended with the squashed-and-stretched object (c) to make contact even more legible.

approach of Chenney et al. [2002]. It consists in scaling the object in its reference frame using the following volume-preserving matrix:

$$\begin{pmatrix} \frac{1}{\sqrt{s}} & 0 & 0 \\ 0 & \frac{1}{\sqrt{s}} & 0 \\ 0 & 0 & s \end{pmatrix},$$

where $s \in [0, 1]$ is a user-controlled parameter in our case.

However, care must be taken to ensure that deformation is applied along the normal of the contact surface, and that the collision point remains unchanged. For the former, we rotate the above squashing matrix so that its Z axis is aligned with the contact surface normal, which we once again approximate with the bisector $\mathbf{n}$. For the latter, we translate the object to leave the collision point $\mathbf{p}_{\text{coll}}$ unaffected by scaling. In practice, we approximate $\mathbf{p}_{\text{coll}}$ with the closest point among object vertices $\mathcal{V}$ in the direction of $\mathbf{n}$:

$$\mathbf{p}_{\text{coll}} = \arg \min_{\mathbf{p}_i \in \mathcal{V}} (\mathbf{p}_i \cdot \mathbf{n}),$$

which is a satisfying approximation for convex objects.

4 Results

We have implemented our trajectory-aware smearing tool on top of the publicly available Blender add-on of Basset et al. [2024]. The add-on is divided in two parts: a python script and a geometry node graph. We distribute computation among these parts with interactive control in mind. For the canceling of rotation, we modified the script to store self-rotation data per object and per frame, and compute vertex trajectories on the fly in the geometry node graph. For curvature-based weighting, we modify the script to compute curvatures for all trajectory points of an object centroid (cusps are flagged as having infinite curvature), and perform the two nested integrals in Equations 3 and 4 through a single repeat zone in the geometry node graph. For collision handling, we compute the bisector $\mathbf{n}$ and collision point $\mathbf{p}_{\text{coll}}$ per contact frame f_{coll} in the script, and apply the corresponding collision smearing and squash-and-stretch transforms interactively using geometry nodes. In supplemental material, we provide scene files for all examples in the paper. For ease of use, scripts have already been applied, but geometry node parameters may be modified at will.

In the following, we illustrate our approach with several test animations (videos are provided in supplemental material).

Jokari. In Figure 1, we compare our approach to the one of Basset et al. [2024], using a scene from the work of Farhat et al. [2025]. Without curvature-based weighting, the deformations at extrema of trajectory curvatures (including the collision) look unnatural (we use the default stylization function with $\beta_{\text{max}} = 1$). Our collision smearing method (here with $m_p = m_n = 2$, $\lambda_{\text{min}} = 0$ and $\lambda_{\text{max}} = 0.7$) increases the legibility of contact, which makes for a “snap” effect when combined with squash-and-stretch (using $s = 0.25$).

Spinning top. The impact of canceling self-rotations is illustrated in Figure 6 on a spinning top animation from Farhat et al. [2025] similar to the example previously given by Basset et al. [2024], using the default stylization function with $\beta_{\text{max}} = 1$. Keeping self-rotations fixed for smearing properly conveys the object’s silhouette motion, while still depicting rotations and avoiding self-intersections. We also show that our approach may be applied to 3D motion lines.

Toy cow. A simple rigid animation of a toy cow along a trajectory with a sharp turn is depicted in Figure 7, which shows curvature-based weighting is required to avoid elongating the object past the sharp turn. Here we use $\beta_{\text{max}} = 2.5$ and $a = 1$.

Fall flat. Our approach may also be applied to an articulated character provided it is treated as a single object. In Figure 8, we reproduce an example from the work of Basset et al. [2024], where the β_{max} parameter has been keyframed with large values to exaggerate the collision of a character falling onto the ground. No such fine-tuning is required with our collision smearing (we use $\beta_{\text{max}} = 1$, $\lambda_{\text{min}} = -0.3$ and $\lambda_{\text{max}} = 0$), with the additional advantage that vertices closer to the contact surface are more displaced, better preserving the character shape and posture. As may be seen in the supplemental video, we do not use squash-and-stretch in this case.

5 Discussion

We have introduced a 3D stylization method that produces elongated smears in a trajectory-aware fashion. It improves upon the approach of Basset et al. [2024] in three respects: it handles self-rotations to mainly convey object’s silhouette motion; it avoids

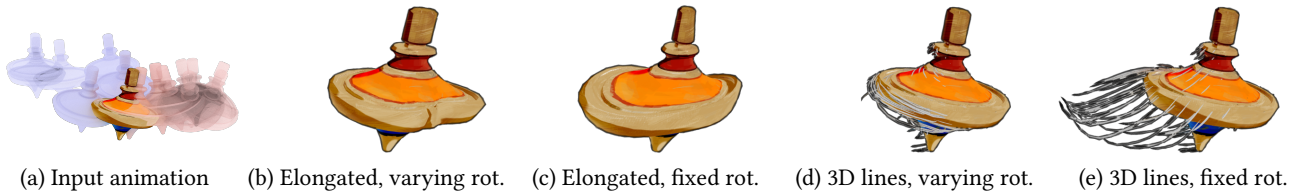


Figure 6: Onion skin visualization of a spinning top animation (a). Using the original self-rotations (b) creates geometric artifacts that hinder motion legibility, which is corrected when keeping the self-rotation of the current frame fixed when smearing (c). We apply the same approach to 3D motion lines [Basset et al. 2024] with varying (d) or fixed (e) self-rotations.



Figure 7: Rigid animation of a toy cow (a). The method of Basset et al. [2024] (b) smears the object past the sharp trajectory turn. Curvature-based weighting (c) solves this issue.

unnatural deformations at sharp trajectory turns through curvature-based weighting; and it conveys contact in an exaggerated manner through a combination of collision smears and squash-and-stretch.

The main current limitation of our approach is that it is tailored to the deformation of single globally rigid objects. A natural extension, in the line of previous work [Basset et al. 2024; Kalyanasundaram et al. 2022; Rohmer et al. 2021], would be to consider articulated characters. Even though self-rotations of the whole character might entail no modification, curvature-based weighting will require to consider more trajectories than merely the centroid trajectory – most likely trajectories of individual bones. Per-bone collision handling should also require more work, perhaps even changes to the skeleton pose [Kwon and Lee 2012] for squash-and-stretch.

Another limitation lies in our approximation of the collision point and normal of the impact plane to compute the squash-and-stretch effect. While this gives satisfying results for globally convex objects, this may cause interpenetration artifacts if the impact occurs inside a concavity. Proper collision detection should be used instead but it would be much more computationally expensive. Note however that our method works as is if the squash-and-stretch effect is done manually by an artist.

An interesting avenue of future work concerns the decomposition of motion. As highlighted by Farhat et al. [2025], smear frames are most useful when they convey the motion of the object’s silhouette in the image plane. Both their method and ours allow for the stylization of a specific subset of transforms; but this requires the motion to be properly decomposed beforehand. In some cases, such as when the animation comes from a complex soft body simulation,



Figure 8: Animation of an articulated character falling on the ground (a). Basset et al. [2024] keyframed smearing parameters by hand (b) to exaggerate motion just before collision. Our collision smearing approach (c) does so automatically, while better preserving the character shape and posture.

motion does not come decomposed this way. The automatic extraction of those transformations that cause changes in an object’s silhouette motion is thus a relevant challenge to tackle in our view.

References

- Jean Basset, Pierre Bénard, and Pascal Barla. 2024. SMEAR: Stylized Motion Exaggeration with ART-direction. In *SIGGRAPH 2024 - Conference & Exhibition on Computer Graphics & Interactive Techniques*. doi:10.1145/3641519.3657457
- Edwin Catmull and Raphael Rom. 1974. A class of local interpolating splines. In *Computer Aided Geometric Design*. Academic Press, 317–326. doi:10.1016/B978-0-12-079050-0.50020-5
- Stephen Chenney, Mark Pingel, Rob Iverson, and Marcin Szymanski. 2002. Simulating cartoon style animation. In *Proceedings of the 2nd International Symposium on Non-Photorealistic Animation and Rendering (NPAR '02)*. ACM, 133–138. doi:10.1145/508530.508553
- Stelian Coros, Sebastian Martin, Bernhard Thomaszewski, Christian Schumacher, Robert Sumner, and Markus Gross. 2012. Deformable objects alive! *ACM Trans. Graph.* 31, 4, Article 69 (2012), 9 pages. doi:10.1145/2185520.2185565
- James E Cutting. 2002. Representing motion in a static image: constraints and parallels in art, science, and popular culture. *Perception* 31, 10 (2002), 1165–1193. doi:10.1068/p3318
- Amine Farhat, Alexandre Bléron, Romain Vergne, and Joëlle Thollot. 2025. Motion ribbons: Parametrized surfaces for depicting motion effects. *Computers and Graphics* 129 (2025). doi:10.1016/j.cag.2025.104227
- Marcos Garcia, John Dingliana, and Carol O’Sullivan. 2007. A Physically Based Deformation Model for Interactive Cartoon Animation. In *Proceedings of the Fourth Workshop on Virtual Reality Interactions and Physical Simulations*. 27–34. doi:10.2312/PE/vrphys/vrphys07/027-034
- Xuezhen Huang, Qiming Hou, Zhong Ren, and Kun Zhou. 2012. Scalable Programmable Motion Effects on GPUs. *Computer Graphics Forum* 31, 7 (2012), 2259–2266. doi:10.1111/j.1467-8659.2012.03219.x
- Nathaniel Jones and John Keyser. 2005. Real-time geometric motion blur for a deforming polygonal mesh. In *International 2005 Computer Graphics*. 26–31. doi:10.1109/CGI.2005.1500362
- Niranjan Kalyanasundaram, Damien Rohmer, and Victor B. Zordan. 2022. Acceleration Skinning: Kinematics-Driven Cartoon Effects for Articulated Characters. In *Proceedings of Graphics Interface 2022 (GI 2022)*. 271 – 279. doi:10.20380/GI2022.26
- Ji-yong Kwon and In-Kwon Lee. 2012. The Squash-and-Stretch Stylization for Character Motions. *IEEE Transactions on Visualization and Computer Graphics* 18, 3 (2012),

- 488–500. doi:10.1109/TVCG.2011.48
- John Lasseter. 1987. Principles of traditional animation applied to 3D computer animation. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*. ACM, 35–44. doi:10.1145/37401.37407
- Jiaju Ma, Li-Yi Wei, and Rubaiat Habib Kazi. 2022. A Layered Authoring Tool for Stylized 3D animations. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. ACM, Article 383, 14 pages. doi:10.1145/3491102.3501894
- Matthias Müller, Bruno Heidelberger, Matthias Teschner, and Markus Gross. 2005. Meshless deformations based on shape matching. *ACM Trans. Graph.* 24, 3 (2005), 471–478. doi:10.1145/1073204.1073216
- Damien Rohmer, Marco Tarini, Niranjan Kalyanasundaram, Faezeh Moshfeghifar, Marie-Paule Cani, and Victor Zordan. 2021. Velocity Skinning for Real-time Stylized Skeletal Animation. *Computer Graphics Forum* 40, 2 (2021). doi:10.1111/cgf.142654
- Johannes Schmid, Robert W. Sumner, Huw Bowles, and Markus Gross. 2010. Programmable motion effects. *ACM Trans. Graph.* 29, 4, Article 57 (2010), 9 pages. doi:10.1145/1778765.1778794
- Richard Williams. 2001. *The animator's survival kit*. Faber and Faber.
- Jiayi Eris Zhang, Seungbae Bang, David I. W. Levin, and Alec Jacobson. 2020. Complementary dynamics. *ACM Trans. Graph.* 39, 6, Article 179 (2020), 11 pages. doi:10.1145/3414685.3417819